

DataFrames with pandas, Polars and Arrow

pandas, eager

Polars, lazy

Arrow, the memory

What are DataFrames and Arrow?

The topic

pandas and **Polars** are two engines that run filter, join and aggregate on a **DataFrame**.

A **DataFrame** is a typed table stored in memory.

Arrow is how the table sits in memory under both.

The question I answer

Why the same query read the whole file in **pandas** and a small part of it in **Polars**?

How can the result move between **pandas** and **Polars** without a copy?

The demo shows

1. The *MeteoSwiss* aggregation in **pandas** and in **Polars**.
2. The timing of the same query in both.
3. How a table is passed from **Polars** to **pandas** through **Arrow**, with no copy.

Plan

- 1 DataFrames** a typed table in memory, and our MeteoSwiss data
- 2 pandas** eager: runs each step in the order you wrote it
- 3 Polars** lazy: plans the whole query, then runs it
- 4 Arrow** the standard memory format, its strengths and limits
- 5 Demo** the same query in both, and a table passed without a copy
- 6 Conclusion** which one to use, and when

What is a DataFrame?

A DataFrame is a table of typed columns, held in memory, including an API to filter, join and aggregate it.

THE DATAFRAME PIPELINE

from a file or a database → into one Python object → held in RAM, not on disk → gone when the session closes

The standard Python library doesn't handle tables natively → we need an external library:



one machine, **eager**

reads everything, then works

single core (has arrow in version 3.0)

+



a cluster, **lazy**

splits work across machines

multi-core

=

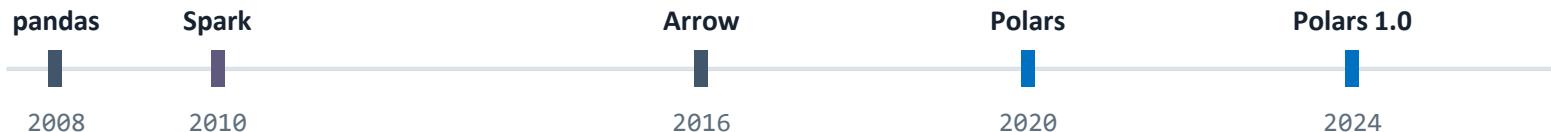


one machine, **lazy**

plans the chain, then runs it

Rust, Arrow-native, multi-core

TECHNOLOGY TIMELINE



DataFrames for different usage

2 Workflows

Data analytics

SOURCES

API, Kafka, Excel, database



Main goal:
study the data

DATAFRAME

explore, clean, combine, aggregate

TARGET

a dashboard, a chart, a number



Analysis stops at a chart

Data engineering (ETL)

SOURCES

API, Kafka, Excel, database



Main goal:
transform the data

DATAFRAME

extract, transform, load

TARGET

API, Kafka, Excel, database



pipeline writes back to a system

2 Historical Tools



Simple 1 machine
Slow



Lazy (optimization)
Simple!



Lazy (optimization), Fast + Massive data
Complex

Underneath, they all speak SQL

The same five operations, renamed. Learn them once and the next library is mostly a rename.

operation	 SQL	 pandas	 Polars
pick columns	SELECT a, b	df[["a", "b"]]	df.select("a", "b")
keep rows	WHERE x > 0	df[df.x > 0]	df.filter(pl.col("x") > 0)
combine tables	JOIN o ON k	df.merge(o, on="k")	df.join(o, on="k")
summarise	GROUP BY k	df.groupby("k").mean()	df.group_by("k").agg(...)
order	ORDER BY x	df.sort_values("x")	df.sort("x")

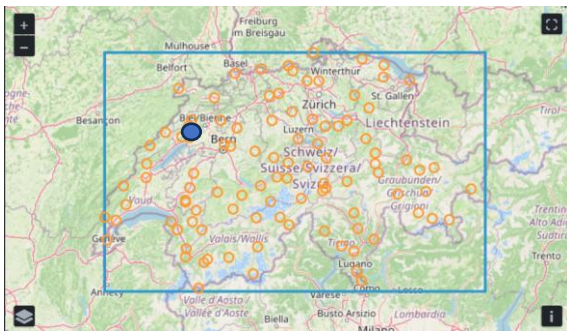
Do you know the difference between a SQL database and a DataFrame?

SQL the table stays in the database, you send a query

DataFrame the table is loaded in RAM, you work on it directly

Get the data

from MeteoSwiss to parquet files



[Automatic weather stations - Measurement values - data.geo.admin.ch](https://data.geo.admin.ch)

station_abbr	ts	temperature	humidity	pressure
str	datetime[μs]	f64	f64	f64
"abo"	2026-03-24 14:40:00	8.8	44.5	870.2
"abo"	2026-03-24 14:50:00	9.1	44.2	870.2
"abo"	2026-03-24 15:00:00	9.4	45.8	870.1

149 stations, one row every 10 minutes, 6 months → 3 837 651 rows

THEN LOAD INTO A DATAFRAME:

DataFrame = the typed table + an integrated API to filter, join, aggregate + an engine that decides when they run

pandas is eager: it runs each step in order

What you write

```
# pandas
measurements = pd.read_parquet(MEASUREMENTS_FILE)
stations = pd.read_parquet(STATIONS_FILE)
joined = measurements.merge(stations,
                             on="station_abbr")

one_station = joined[
    joined["station_abbr"] == STATION]
result = monthly_mean(one_station) # groupby
```

What pandas runs

share of the rows

read_parquet	every station, every column	100 %
merge	join every row to its station	100 %
filter	keep "ber"	≈ 0.7 %
groupby	7 rows out	7 rows

3.8 million rows read and joined, to keep one station.
Eager: pandas will not optimize. It is your job to move the filter up.

Polars is lazy: it plans before executing

What you write

```
# Polars
measurements = pl.scan_parquet(MEASUREMENTS_FILE)
stations = pl.scan_parquet(STATIONS_FILE)
plan = (measurements
        .join(stations, on="station_abbr")
        .filter(pl.col("station_abbr") == STATION)
        .group_by(pl.col("ts").dt.strftime("%Y-%m"))
        .agg(pl.col("temperature").mean()))
result = plan.collect()
```

What Polars runs (plan.explain())

```
INNER JOIN:
LEFT PLAN ON: [col("station_abbr")]
  Parquet SCAN [data/measurements.parquet]
  PROJECT 3/5 COLUMNS
  SELECTION: col("station_abbr") == "ber"
  ESTIMATED ROWS: 3831715
RIGHT PLAN ON: [col("station_abbr")]
  Parquet SCAN [data/stations.parquet]
  PROJECT 3/4 COLUMNS
  SELECTION: col("station_abbr") == "ber"
  ESTIMATED ROWS: 149
END INNER JOIN
```

scan_parquet returns a plan. Nothing runs before **collect()**.

What Polars changed in the plan

Predicate pushdown **SELECTION**

the filter runs while reading: row groups without "ber" are skipped

Projection pushdown **PROJECT 3/5**

only 3 of the 5 columns are read from the file

What is Arrow?

Apache Arrow (2016) is a standard format to store a table in memory, column by column.

Any tool using this standard can read the same bytes, without converting or copying them.

pandas

NumPy arrays by default. Arrow columns only through pyarrow (pd.ArrowDtype).

pyarrow

the Python library of Arrow. `to_arrow()`, `to_pandas()` and `from_pandas()` use it.

Polars

Polars has its own Arrow implementation, written in Rust.

The shared layout in memory

station_abbr
 offsets + utf8

ts
 int64 [μs]

temperature
 float64

humidity
 float64

pressure
 float64

one contiguous buffer per column: the CPU reads a whole column in one go

a validity bitmap: to mark missing values

a schema: every tool knows the type of each column without guessing

Each library keeps its own implementation. The table goes from one to the other through pyarrow, and the numbers are not copied.

Arrow: strengths and limits

Strengths

Shared, not copied

pandas (with Arrow columns), Polars, DuckDB and Spark read the same bytes. (see demo)

Fast to scan

each **column is one block in memory**,
so the CPU reads a whole column in one go.

Text in one block

NumPy keeps one Python object per string.
Arrow packs the whole column.

Missing values for any type

a **validity bitmap** marks them. NumPy only has NaN, and only for decimals.

Limits

A format, not an engine

Arrow stores the table. Filtering and joining still need pandas, Polars or DuckDB.

Blocks are read-only

changing one value means building a new column.

Not every column is shared

text is converted between Polars and pandas
(take more time, see demo)

pandas needs pyarrow

pandas has no Arrow.
Even pandas 3 keeps NumPy by default
→ we need **pyarrow** for Arrow columns.

Demo

```
$ jupyter lab 1_demo.ipynb # github.com/grey-park/pandas-Polars-Arrow
```

1. pandas, eager

read, merge, filter, groupby

2. Polars, lazy

the plan before and after optimisation

3. one measured number

median of 10 runs, memory held, same answer

4. the Arrow hand-off

one buffer address at every hop

The numbers

Section 3: the same query, median of 10 runs

```
pandas naive  1.371 s    best 1.341 s
pandas tuned   0.132 s    best 0.124 s
Polars lazy    0.010 s    best 0.009 s
```

```
same answer from all three: True
tuning pandas by hand: 10x faster
Polars: 14x faster again
```

Section 4: Polars -> pandas -> Polars, median of 10 runs

	whole table	without text
Arrow columns:	167.2 ms	31.3 ms
NumPy columns:	248.2 ms	68.1 ms
copy costs	1.5x	2.2x

Section 4: address of the temperatures at each step

Polars -> Arrow	0x1d97df30000	
Arrow -> pandas (Arrow)	0x1d97df30000	same: True
Arrow -> pandas (NumPy)	0x1d9bb3d0000	same: False
pandas -> Polars	0x1d97df30000	same: True

Operation by operation

2_benchmark.ipynb: each operation alone, best of 10 runs, Polars in lazy mode (collect() included)

  7 rows ▾ 7 rows × 4 cols				
operation	123 pandas (ms)	123 Polars (ms)	123 ratio	
"read Parquet"	116.72	36.12	3.2	
"select columns"	0.92	0.03	30.7	
"filter rows"	29.02	2.56	11.3	
"join stations"	1069.26	41.75	25.6	
"group + mean"	92.59	11.49	8.1	
"sort"	348.77	45.81	7.6	
"write Parquet"	635.84	89.41	7.1	

Conclusion

pandas runs each step as written.

Easy to read and debug. Use it to **explore data** that fits in memory.

Polars plans the whole chain, then runs it.

Most of the file never leaves the disk. Use it when **efficiency** is required

Arrow is the memory under both.

Passing a table costs almost nothing

Questions?

Sources

McKinney (2010). Data structures for statistical computing in Python. Proc. SciPy. doi:10.25080/Majora-92bf1922-00a

Ritchie Vink (2023). Polars: DataFrames in the multi-core era. PyData NYC. youtube.com/watch?v=tqcudsykOGc

Apache Arrow. A deep dive into the Arrow Columnar format with pyarrow and nanoarrow. youtube.com/watch?v=k0mkDXWfLb4

Pandas vs PySpark vs Polars: The DataFrame Explained Visually. youtube.com/watch?v=TYnXqVrVz8k

DuckDB vs Pandas vs Polars For Python devs. youtube.com/watch?v=4DloACFItec

Polars docs. Lazy API : optimizations: docs.pola.rs/user-guide/lazy/optimizations

pandas docs. PyArrow functionality: pandas.pydata.org/docs/user_guide/pyarrow.html

MeteoSwiss. SwissMetNet open data: data.geo.admin.ch/ch.meteoschweiz.ogd-smn (CC-BY)